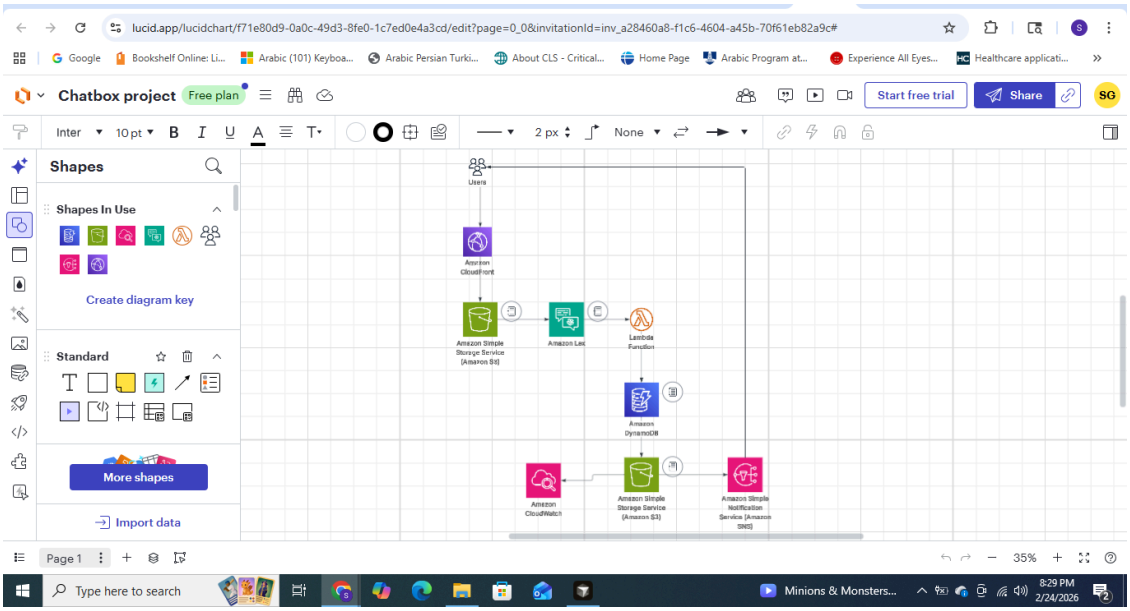
Voter Assistant Chatbot Project

Objective: With millions of Texans represented in upcoming senatorial elections, many voters remain unaware of where candidates stand on key policy issues. To address this gap, I developed a cloud-based, AI-integrated voter assistant chatbot designed to help inform constituents in the Dallas–Fort Worth area. By collecting user preferences on major political issues and comparing them with candidate positions, the system recommends candidates whose stances most closely align with the voter's views. The goal of this project is to improve voter awareness and encourage more informed participation as primary elections approach. I went with a serverless architecture for the chatbot, eliminating the need for constant instance maintenance while also saving substantially on costs to operate

Skills/Tools utilized:

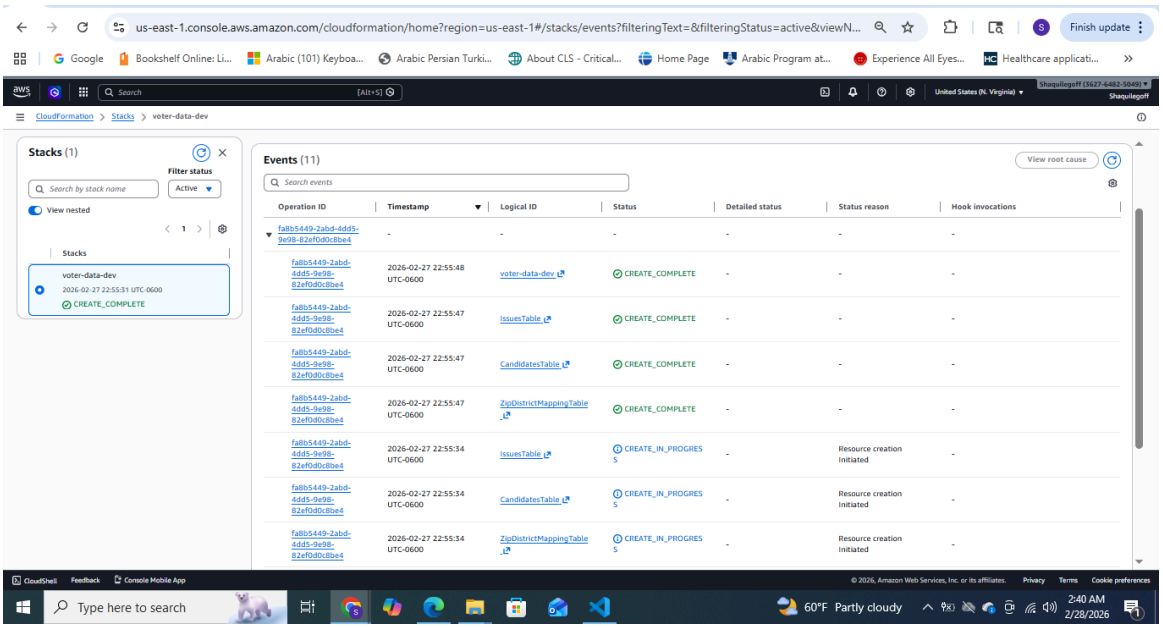
Serverless Architecture
AWS Lambda
Amazon Lex
AWS Cloudformation
IAM
AWS CloudFront
Amazon S3
Route53 (future)
SES (future)
Lucid

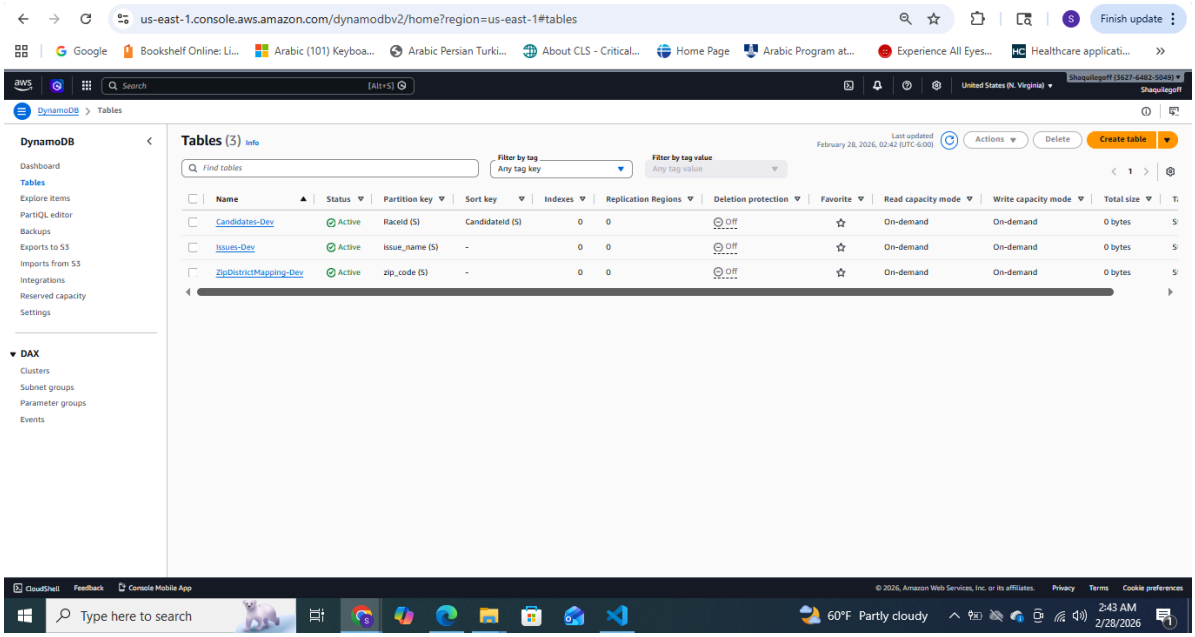
Steps:
My first step was creating a rough sketch of the cloud architecture, which I created in Lucid.



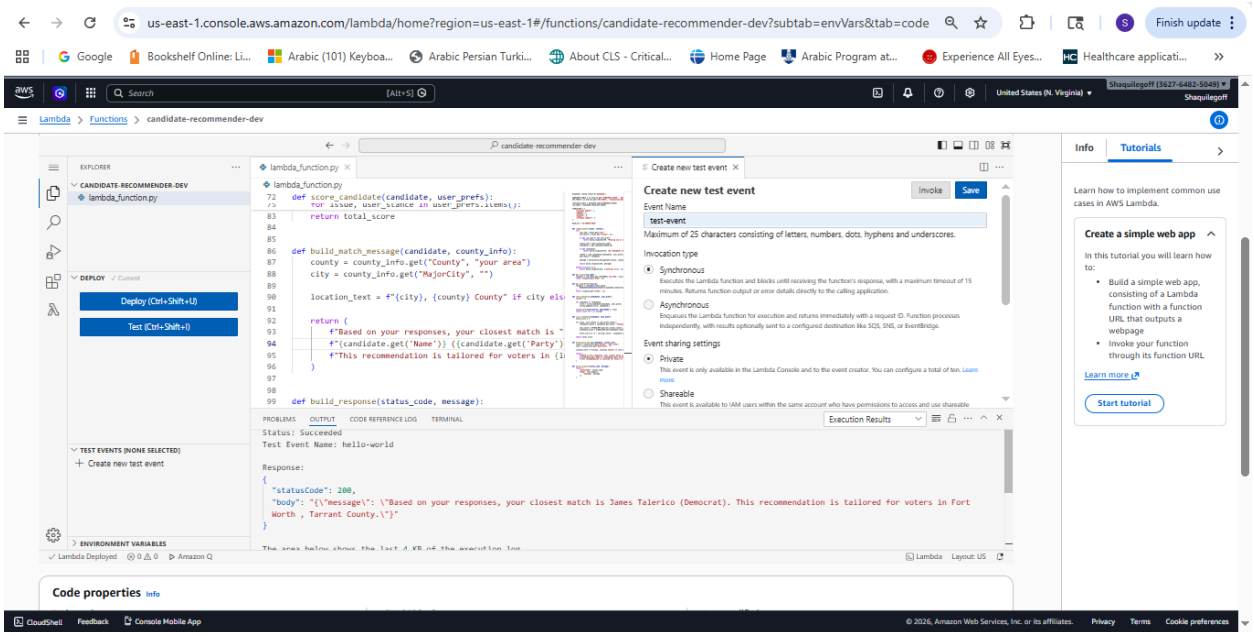
I intend to have the users access the application through a Content Delivery Network (CloudFront) to reduce latency, while also providing an added layer of security protection against DDoS attacks, potentially integrating WAF as well. The CDN would sit in front of an S3 bucket hosting the static website, with Lex embedded. User input to Lex would then trigger the lambda function, which would then access DynamoDB in the database layer to pull relevant candidate info, calculate alignment scores, and return a response to the user. A separate S3 bucket with relevant PDFs of candidate information would also be created to provide more context for Lex and the user. CloudWatch would be used to monitor resource logs and set alarms to ensure cost compliance and proper resource allocation. Lastly, I may create a subscription in SNS to send users reminders of upcoming elections via email.

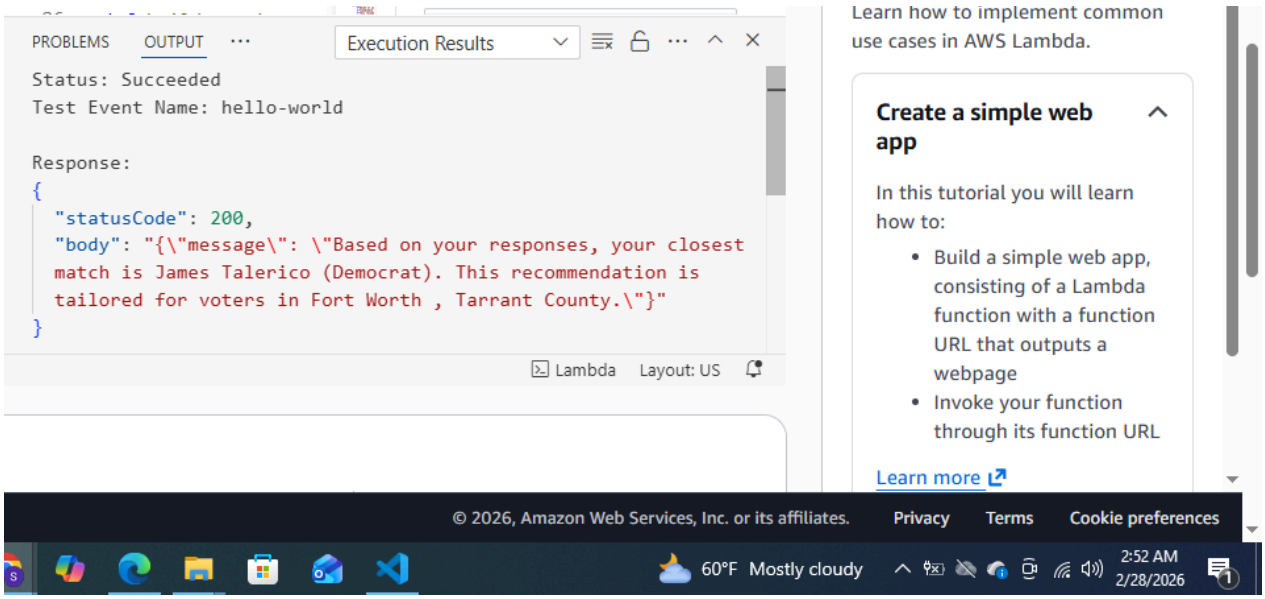
After creating the design flow, I provisioned the data layer using CloudFormation. I created the stack for the database layer and seeded relevant info like candidates' stances, political parties and polling locations based on inputted user location.





Next, I setup my Lambda function, configuring an IAM role to allow DynamoDB access. Lambda allowed me to run a serverless application, without the need to provision entire ec2 instances.

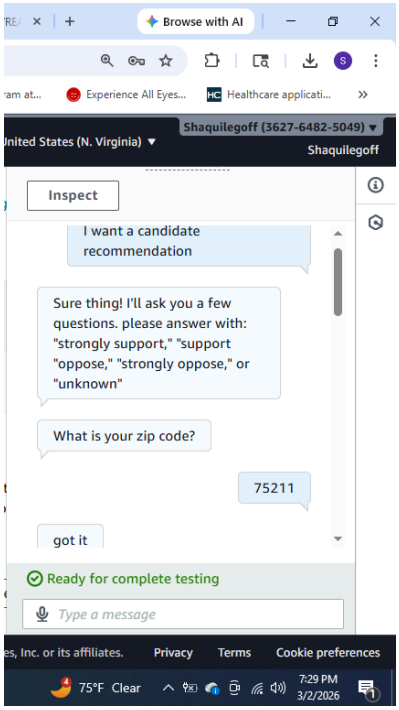


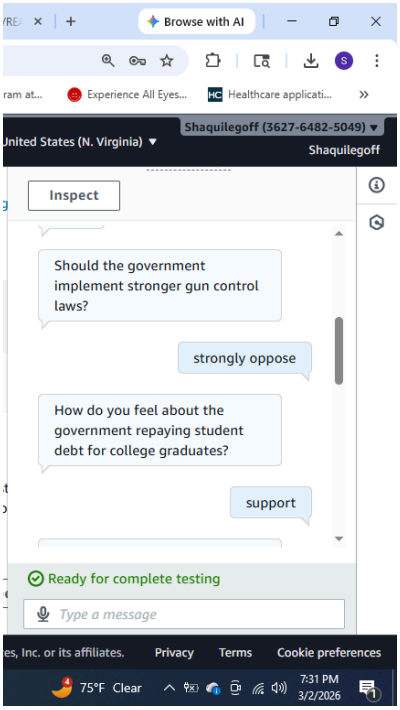


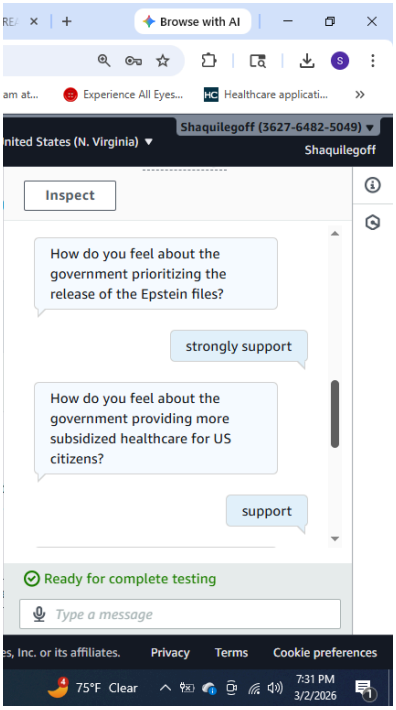
Once I determined that Lambda was fully functional through iterative debugging and testing, my next goal was to integrate Lex. I configured Lex, adding in utterances and intents. This would allow the user to engage in conversational flow with the chatbot, responding to questions regarding their opinion on controversial issues in US politics. Once finished, the chat box would invoke the previously made Lambda function to pull a score comparing the answers given by

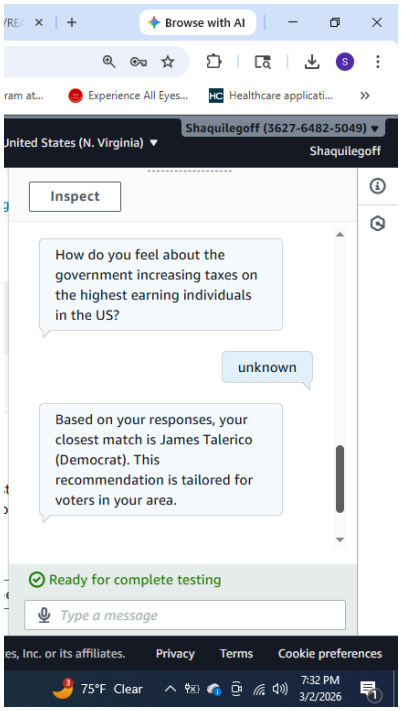
the user with the stances of the candidate. Below are some sample conversational flows from the chatbot:

Sample 1

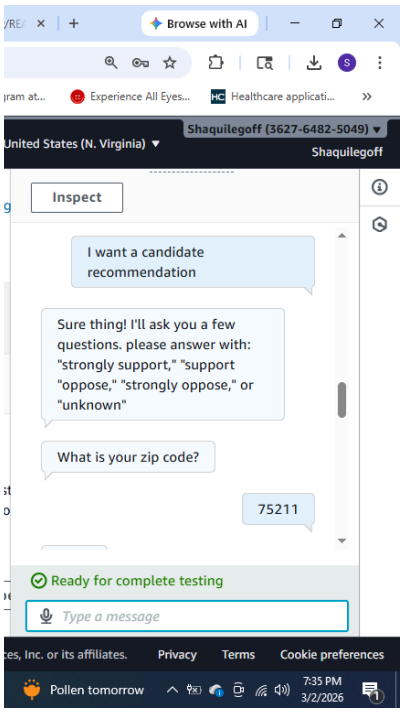


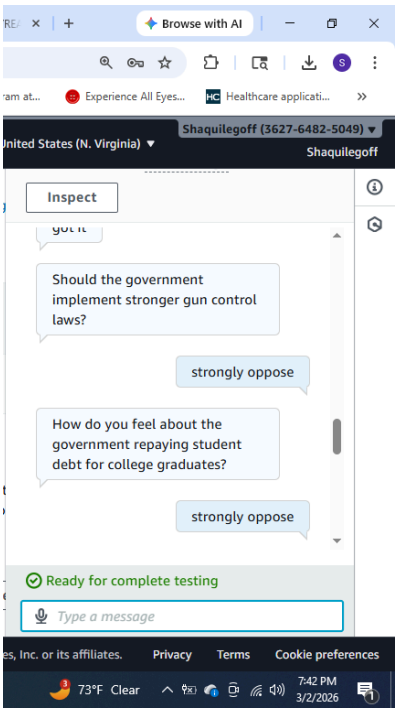


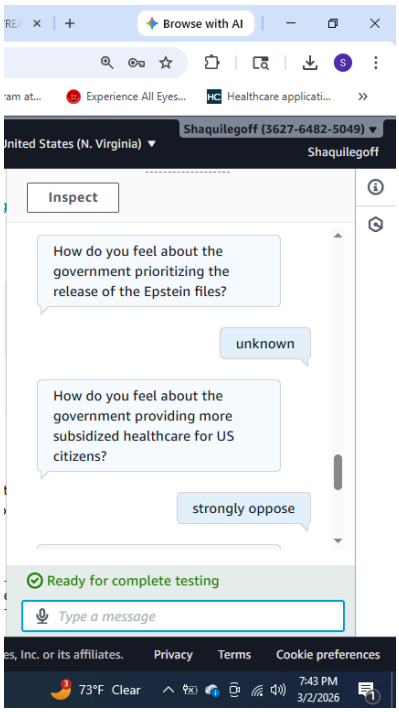


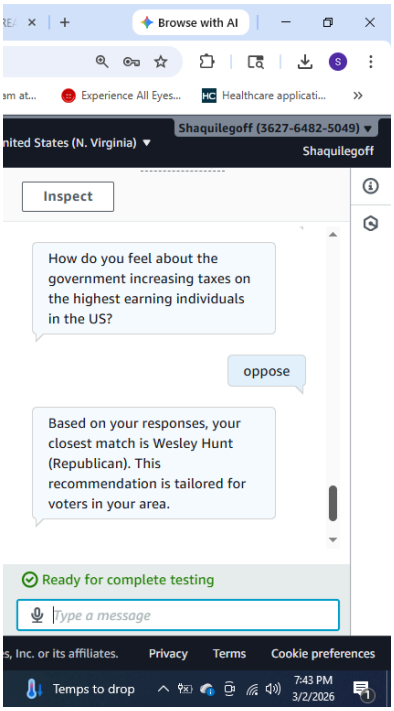


Sample 2

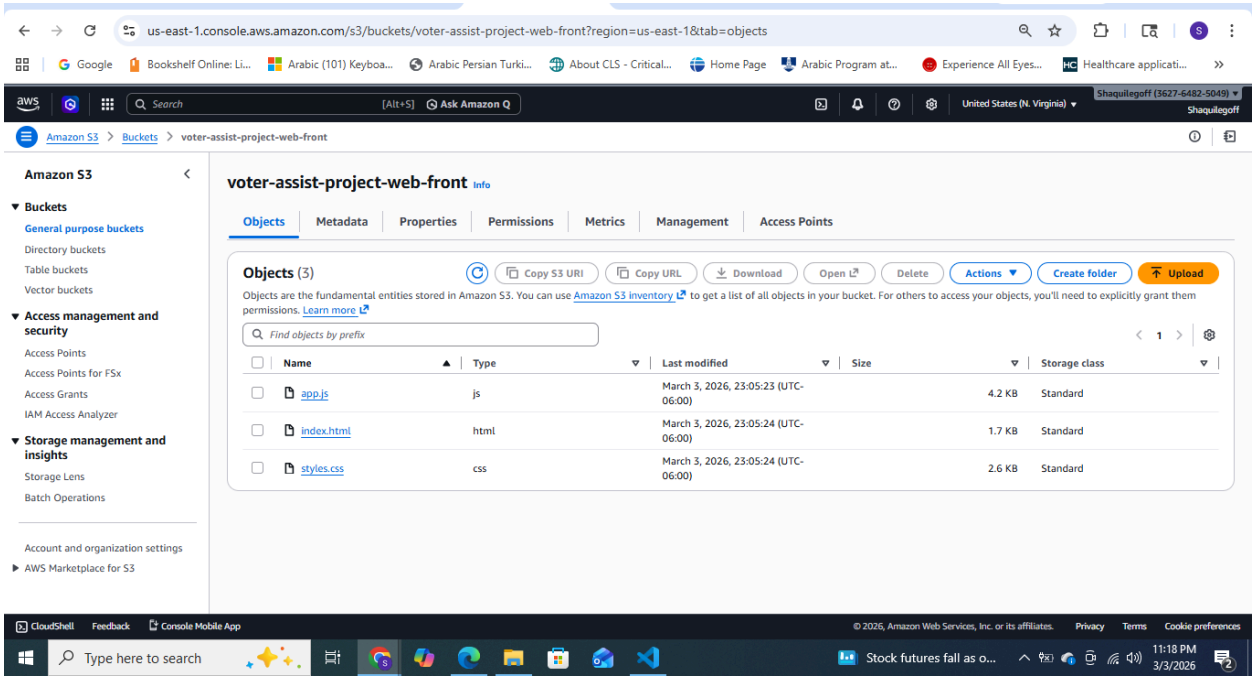

Browse with AI
Experience All Eyes...
Healthcare applicati...
Shaquilegoff (3627-6482-5049)
United States (N. Virginia)
Shaquilegoff
Inspect
I want a candidate recommendation
Sure thing! I'll ask you a few questions. please answer with: "strongly support," "support "oppose," "strongly oppose," or "unknown"
What is your zip code?
75211
Ready for complete testing
Type a message
Privacy
Terms
Cookie preferences
Pollen tomorrow
7:35 PM
3/2/2026


Browse with AI
Experience All Eyes...
Healthcare applicati...
Shaquilegoff (3627-6482-5049)
United States (N. Virginia)
Shaquilegoff
Inspect
Should the government implement stronger gun control laws?
strongly oppose
How do you feel about the government repaying student debt for college graduates?
strongly oppose
Ready for complete testing
Type a message
Privacy
Terms
Cookie preferences
73°F Clear
7:42 PM
3/2/2026

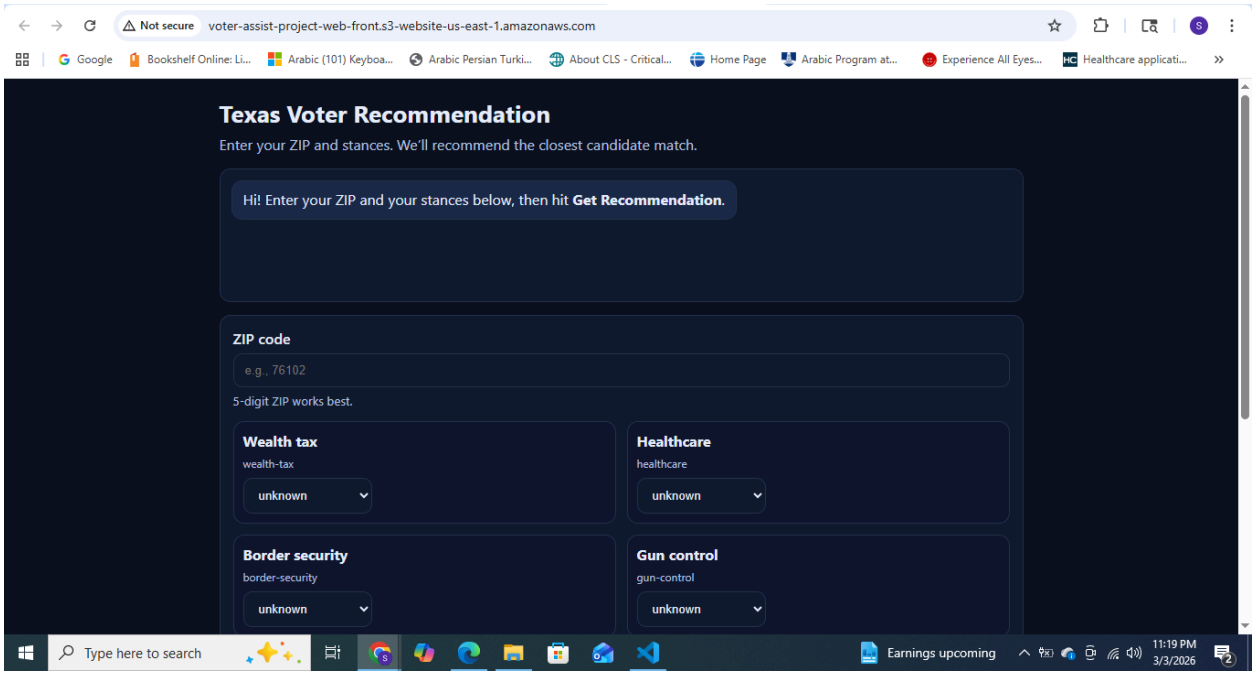

Browse with AI
Experience All Eyes...
Healthcare applicati...
Shaquilegoff (3627-6482-5049)
United States (N. Virginia)
Shaquilegoff
Inspect
How do you feel about the government prioritizing the release of the Epstein files?
unknown
How do you feel about the government providing more subsidized healthcare for US citizens?
strongly oppose
Ready for complete testing
Type a message
Privacy
Terms
Cookie preferences
73°F Clear
7:43 PM
3/2/2026


Browse with AI
Experience All Eyes...
Healthcare applicati...
Shaquilegoff (3627-6482-5049)
Shaquilegoff
Inspect
How do you feel about the government increasing taxes on the highest earning individuals in the US?
oppose
Based on your responses, your closest match is Wesley Hunt (Republican). This recommendation is tailored for voters in your area.
Ready for complete testing
Type a message
Privacy
Terms
Cookie preferences
Temps to drop
7:43 PM
3/2/2026

In a real-world environment, services like Amazon Connect and Bedrock could also be implemented to provide LLM capabilities and an omnichannel environment for a more dedicated user assistance experience. After finishing with my backend, I created an S3 bucket and hosted a static website to allow for public access and usage of my end product.

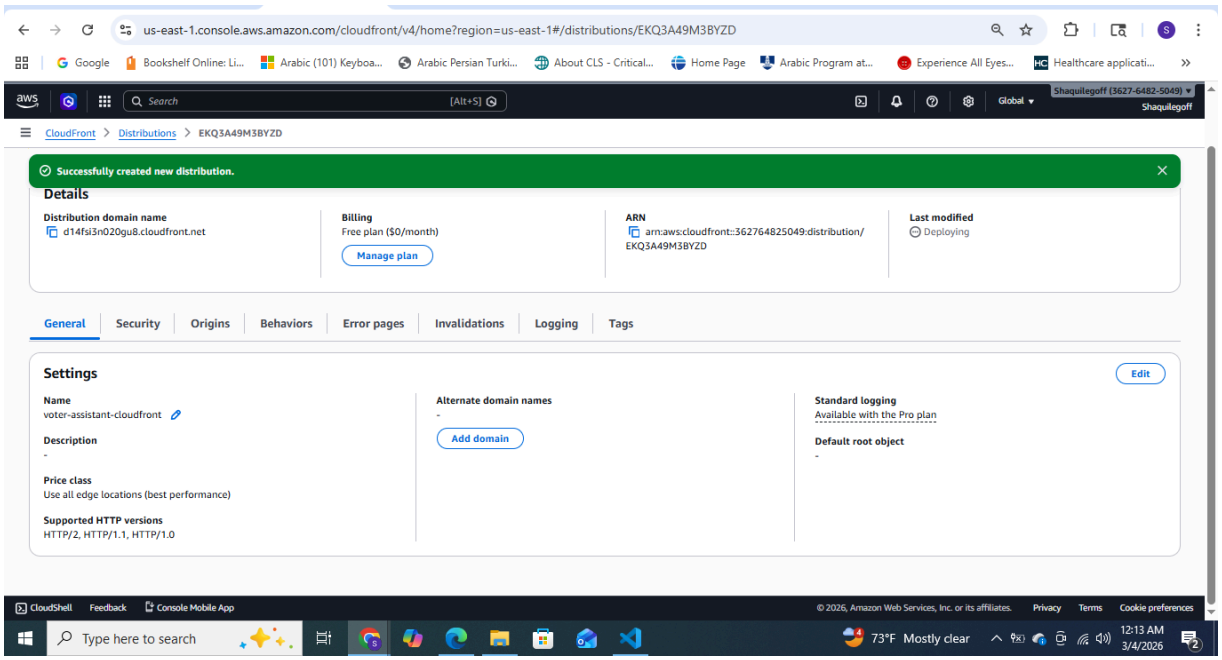


Website frontend:

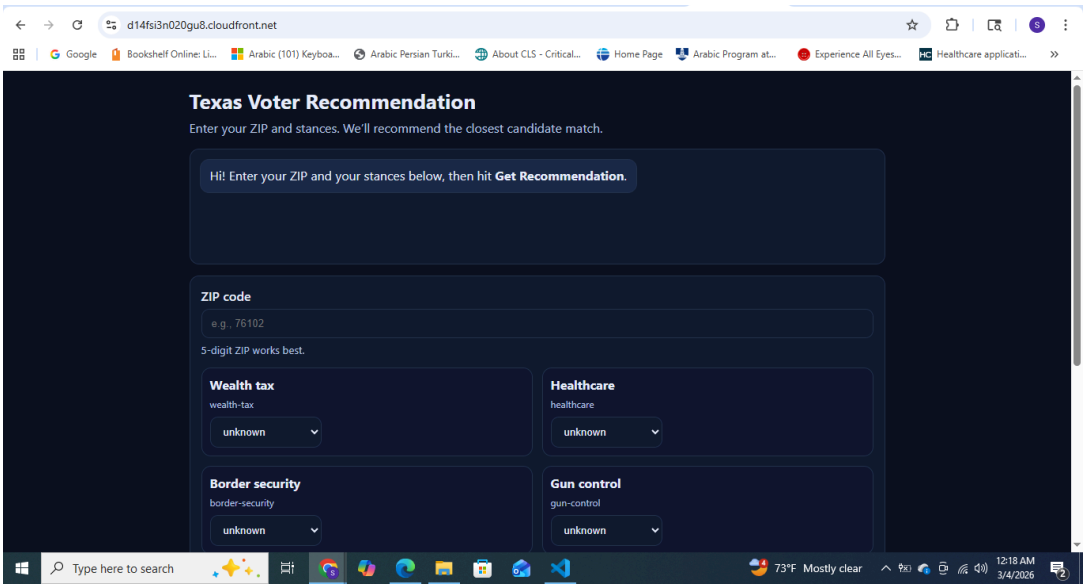


Although the website is now successfully deployed, I wanted to place the s3 bucket behind CloudFront. CloudFront is a great feature because as a Content Delivery Network, it allows content to be delivered quickly by storing cached files in global edge locations, lowering latency. It also has the added security benefit, through AWS Shield, of protecting the infrastructure against malicious activity, including layer 3 and 4 DDoS attacks. If I wanted additional security, which would be recommended in an enterprise environment, I could also upgrade to Shield Advanced or the CloudFront Business plan, which protects at the application level (layer 7) as well.

CloudFront distribution created:



Successful connection to the front end via CloudFront



Future features:
I plan to add Amazon SES, an informational page for each candidate stored in an additional S3 bucket, and a Route53-configured domain name. SES would enable email reminders for relevant elections after collecting user email addresses for a subscription list. The additional S3 bucket would allow users to be better informed about their selected candidate. Lastly, a Route53 configuration would move the application from a hobby project to a real-life, usable tool for voter information. The serverless architecture I utilized also allows for the list of candidates and specific election races to be adjusted and scaled as needed, providing a one-size-fits-all solution for unbiased and informative voting.

Challenges overcome:
This project's challenges were primarily misconfigurations I made while trying to connect the different services. I had to adjust DynamoDB attribute names and comply with DynamoDB's case sensitivity to fix incorrect candidate recommendations. I also had an issue with the API gateway, which I fixed by creating separate branches in my Lambda handler for Lex and the API gateway. Lastly, I faced an issue configuring CORS to allow communication between CloudFront and the API gateway and Origin Access Control to allow communication with the S3 hosted webfront while keeping the S3 bucket private. I resolved these issues by analyzing CloudWatch logs and implementing iterative testing to ensure all components were working, delivering the final successful project.

Conclusion:
This project was my first attempt at building a fully functional web-front application and gave me great insight into a DevOps environment. While I wanted to primarily focus on cloud architecture and systems operation, it was still valuable to see and work through the process of developing a working code to implement in my framework. I also gained valuable experience in serverless architecture. Using AWS managed services like Lambda, Lex, and DynamoDB allowed me to implement a lightweight system with minimal need for continuous upkeep and maintenance of a network comprised of EC2 instances, a design strategy that is invaluable in a fast-paced enterprise when time and cost are the two primary drivers.